# CRICKET TICKET BOOKING SYSTEM

Python Programming Assignment

**Suggested total marks:** 20 | **Basic functionality:** 15 | **Bonus challenges:** 5

## Assignment Scenario

You are required to develop a **Cricket Ticket Booking System** using Python. A cricket stadium has a limited number of tickets available for a cricket match. Each ticket contains a unique seat number, seat name, and booking status.

Initially, all tickets are available for booking. When a customer purchases a ticket, the system must update its status from `Available` to `Sold`.

Your task is to develop a menu-driven program that allows users to view available tickets, purchase tickets, and check the booking status of seats.

## 1. Initial Ticket Data

Create a list containing the following ticket information:

| Ticket Index | Seat Number | Seat Name | Status |
|---|---|---|---|
| 0 | 101 | VIP | Available |
| 1 | 102 | VIP | Available |
| 2 | 103 | General | Available |
| 3 | 104 | General | Available |
| 4 | 105 | Premium | Available |
| 5 | 106 | Premium | Available |
| 6 | 107 | General | Available |
| 7 | 108 | VIP | Available |

You must store all ticket information inside a Python list.

### Suggested data structure:

```
tickets = [
    [101, "VIP", "Available"],
    [102, "VIP", "Available"],
    [103, "General", "Available"],
    [104, "General", "Available"],
    [105, "Premium", "Available"],
    [106, "Premium", "Available"],
    [107, "General", "Available"],
    [108, "VIP", "Available"]
]
```

**Note:** The ticket index is the list position (0–7). It is not stored as a separate value in each ticket record.

## 2. Functional Requirements

### Task 1: Display All Tickets

• Display all tickets.

• Show the seat number, seat name, and booking status.

• Use a loop to traverse the ticket list.

### Task 2: Book a Ticket

• Ask the customer to enter their desired seat number.

• Search for the entered seat number in the list.

• If the seat exists and its status is `Available`, book the ticket.

• Change its status to `Sold`.

• Display a successful booking message.

• If the seat is already sold, display an appropriate message.

• If the seat number does not exist, display an invalid seat number message.

### Task 3: Display Available Tickets

• Display only those tickets whose status is `Available`.

• Do not display sold tickets in this section.

• Use a loop and conditional statement.

### Task 4: Display Sold Tickets

• Display only those tickets whose status is `Sold`.

• Show the seat number and seat category.

### Task 5: Calculate Remaining Tickets

• Count the total number of available tickets.

• Display the number of tickets remaining for booking.

## 3. Menu-Driven Interface

Your program must repeatedly display the following menu:

```
================================
       CRICKET TICKET SYSTEM
================================

1. Display All Tickets
2. Book a Ticket
3. Display Available Tickets
4. Display Sold Tickets
5. Remaining Tickets
6. Exit

Enter your choice:
```

The program should continue running until the user selects option 6.

## 4. Sample Execution

Example program execution

### Customer selects option 2: Book a Ticket

Successful booking

```
Enter seat number: 103

Ticket Booked Successfully!

Seat Number: 103
Seat Name: General
Status: Sold
```

Attempt to book the same ticket again

```
Enter seat number: 103

Sorry! This ticket has already been sold.
```

Remaining ticket calculation

```
Total Tickets: 8
Sold Tickets: 1
Available Tickets: 7
```

## 5. Instructions and Restrictions

1. Use Python lists to store ticket information.
2. Use loops (`for` or `while`) to traverse the list.
3. Use conditional statements (`if`, `elif`, `else`) wherever required.
4. Use a `while` loop to keep the menu running.
5. Use a loop to search for the seat number entered by the customer.
6. Update the original list when a ticket is booked.
7. Do not create a separate list for sold tickets.
8. Do not use dictionaries, classes, or external libraries.
9. Do not use built-in sorting or searching functions.
10. Handle invalid inputs and already-sold tickets appropriately.

## 6. Additional Challenges (Bonus Marks)

After completing the basic system, attempt these additional features:

• **Bonus 1:** Ask the customer how many tickets they want to purchase in one transaction.

• **Bonus 2:** Display the total number of sold tickets.

• **Bonus 3:** Prevent the customer from booking more tickets than are available.

• **Bonus 4:** Display the percentage of tickets sold.

## 7. Learning Objectives

By completing this assignment, students should demonstrate their understanding of:

• One-dimensional and nested lists.

• List indexing and updating values.

• `for` and `while` loops.

• Nested loops (where required).

• Conditional statements.

• Searching elements in a list.

• Counting elements based on conditions.

• Building a menu-driven application.

## 8. Submission Requirements

• Submit the complete Python source code (`.py` file).

• Include screenshots of at least three successful test cases.

• Demonstrate successful booking, duplicate booking prevention, and remaining ticket calculation.

## Important Reminder

> **The assignment is not just about displaying tickets. The main objective is to ensure that the original ticket list is updated whenever a customer purchases a ticket, and that the updated status is reflected throughout the system.**